



Exercício 3.4: Pod multi-container com initContainer, sidecar e emptyDir

Aluno: Luiz Brito · Data: 18/09/2026 · Disciplina: Cloud Native Development (CP1)

Ambiente

- Cluster local **kind** v0.32.0, Kubernetes **v1.36.1**, contexto **kind-fiap-store**, namespace **store-dev**. O sidecar nativo (**initContainers** com **restartPolicy: Always**) exige Kubernetes 1.29 ou superior.
- Imagens: **store-api:1.0** (local, carregada no nó), **busybox:1.36**, **curlimages/curl:8.10.1** e **nicolaka/netshoot**.

Manifestos

01-pod-multi.yaml

```
apiVersion: v1
kind: Pod
metadata:
  name: store-api-full
  namespace: store-dev
  labels:
    app: store-api-full
    tier: backend
    env: dev
spec:
  securityContext:
    runAsNonRoot: true
    runAsUser: 10001
  initContainers:
    - name: preparar-logs
      image: busybox:1.36
      command:
        - sh
        - -c
        - echo "$(date) [preparar-logs] arquivo de log criado" > /var/log/app/access.log
```

```

    volumeMounts:
      - name: logs
        mountPath: /var/log/app
  - name: log-shipper
    image: busybox:1.36
    restartPolicy: Always
    command: ["tail", "-F", "/var/log/app/access.log"]
    volumeMounts:
      - name: logs
        mountPath: /var/log/app
containers:
  - name: store-api
    image: store-api:1.0
    imagePullPolicy: IfNotPresent
    command: ["sh", "-c"]
    args:
      - uvicorn main:app --host 0.0.0.0 --port 8000 2>&1 | tee -a /var/log/app/access.log
    ports:
      - name: http
        containerPort: 8000
    readinessProbe:
      httpGet:
        path: /ready
        port: http
      initialDelaySeconds: 3
      periodSeconds: 5
    livenessProbe:
      httpGet:
        path: /health
        port: http
      initialDelaySeconds: 10
      periodSeconds: 10
    resources:
      requests:
        cpu: 100m
        memory: 128Mi
      limits:
        cpu: 500m
        memory: 256Mi
    volumeMounts:
      - name: logs
        mountPath: /var/log/app
volumes:
  - name: logs
    emptyDir: {}

```

Pontos do desenho:

- `preparar-logs` é o `initContainer` clássico: cria `/var/log/app/access.log` com uma linha de marcação com a data e sai com código 0.
- `log-shipper` é o sidecar moderno: está em `initContainers` com `restartPolicy: Always` e roda `tail -F` indefinidamente.
- `store-api` é o container principal. O `command` / `args` manda a saída do `uvicorn` (`stdout` e `stderr`) para o `tee -a`, que grava no arquivo compartilhado e mantém a saída em `stdout` (o `kubectl logs` continua funcionando).
- `emptyDir logs` montado em `/var/log/app` nos três containers.
- O `securityContext` no nível do Pod (`runAsUser: 10001`, `runAsNonRoot: true`) faz o arquivo criado pelo `init` pertencer ao mesmo usuário que a aplicação usa para acrescentar linhas. Sem isso o `init` (`root` no `busybox`) criaria um arquivo que o `store-api` não conseguiria escrever.

02-svc-multi.yaml

```
apiVersion: v1
kind: Service
metadata:
  name: store-api-full
  namespace: store-dev
spec:
  type: ClusterIP
  selector:
    app: store-api-full
  ports:
    - port: 80
      targetPort: http
```

Evidências

Aplicação

```
$ kubectl apply -f 01-pod-multi.yaml -f 02-svc-multi.yaml
pod/store-api-full created
service/store-api-full created
$ kubectl wait --for=condition=Ready pod/store-api-full -n store-dev --timeout=120s
pod/store-api-full condition met
```

(a) READY com a contagem correta

```
$ kubectl get pod store-api-full -n store-dev
```

NAME	READY	STATUS	RESTARTS	AGE
store-api-full	2/2	Running	0	18s

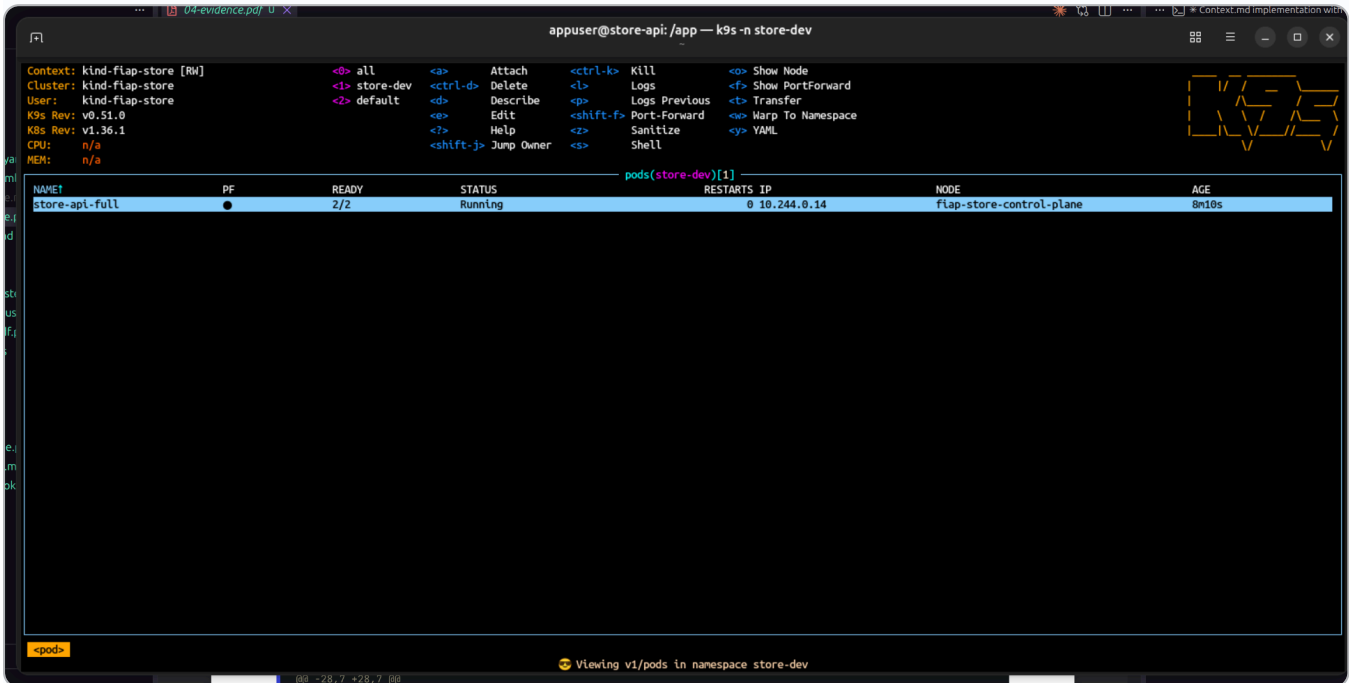
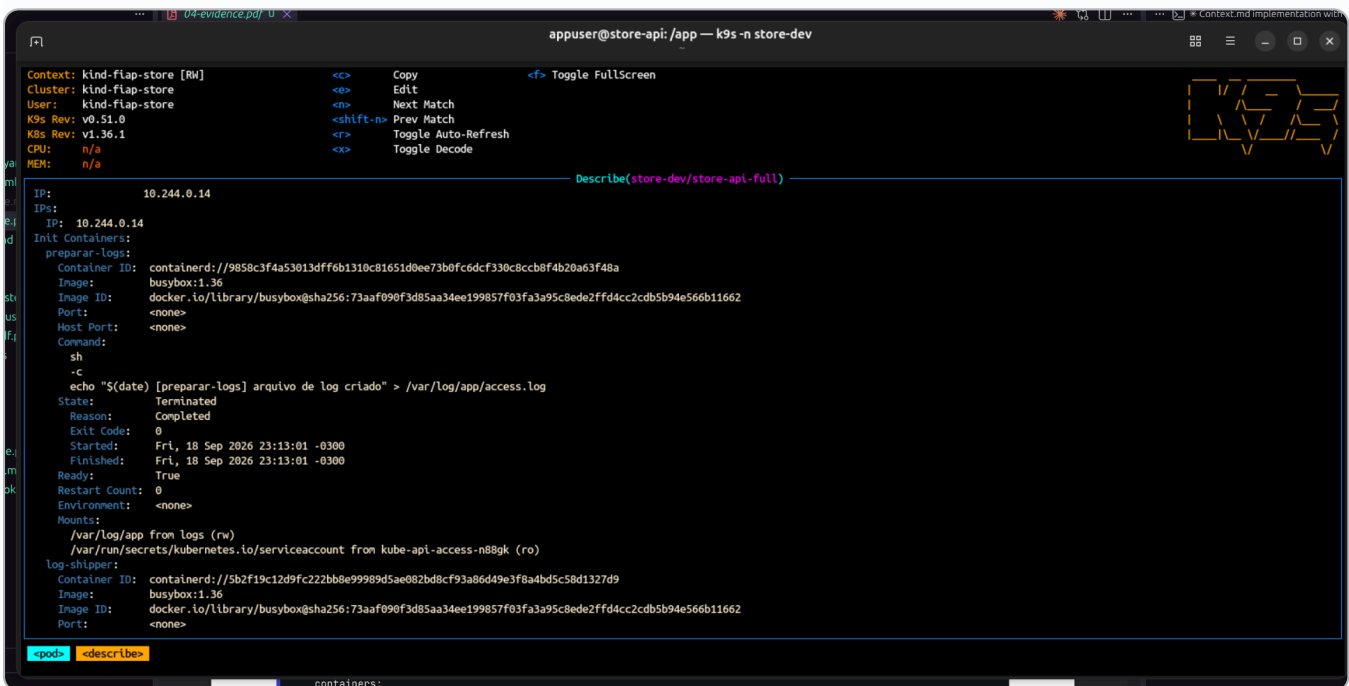


Figura 1 (k9s): `store-api-full` com `READY 2/2` e `Running`.

Por que o sidecar conta como container pronto. O `2/2` é o container principal mais o sidecar; o initContainer clássico `preparar-logs` já terminou e não entra na conta. Como o `log-shipper` tem `restartPolicy: Always`, o kubelet o trata como um container de longa duração, que fica rodando durante toda a vida do Pod, e o inclui na contagem de READY. Sem probes, ele é considerado pronto assim que inicia, e a condição `Ready` do Pod passa a depender dele.

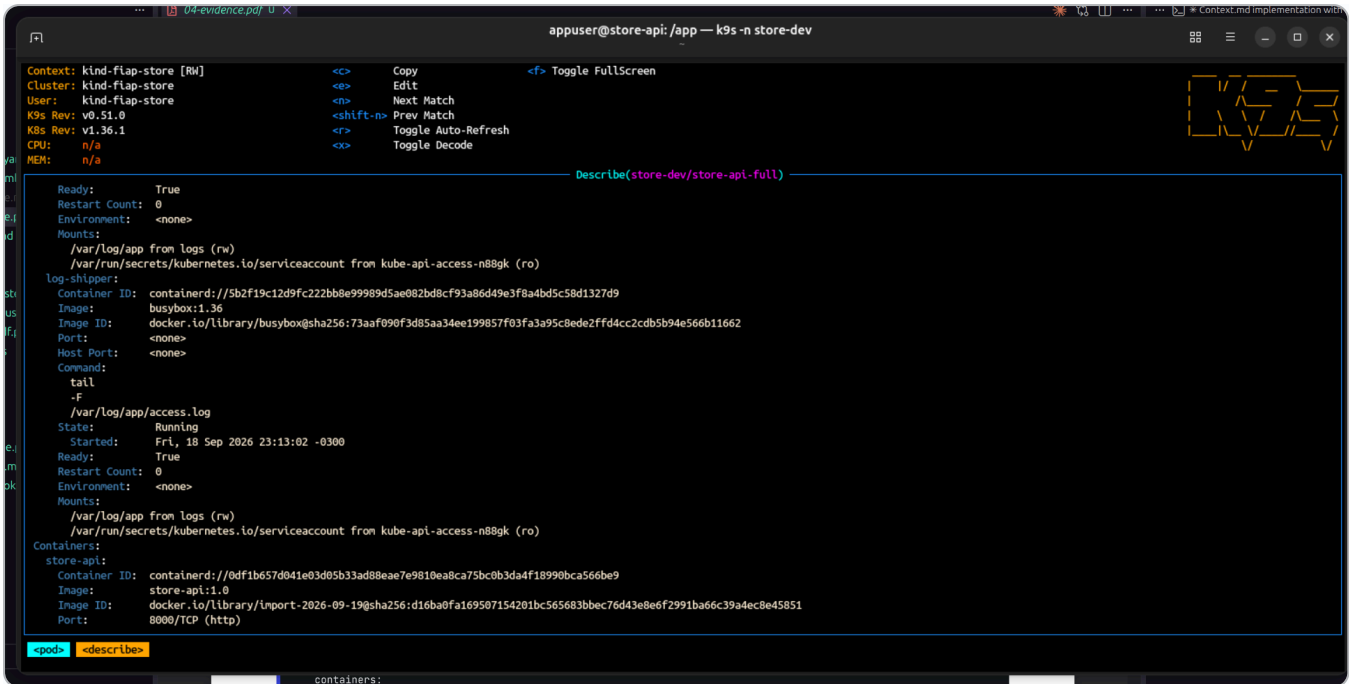
(b) preparar-logs terminado e log-shipper rodando

```
$ kubectl describe pod store-api-full -n store-dev | sed -n '/^Init Containers:$/,/^Containers:/' |  
grep -E '^  [a-z]|State:|Reason:|Exit Code:|Ready:|Containers:'  
Init Containers:  
  preparar-logs:  
    State:      Terminated  
    Reason:     Completed  
    Exit Code:  0  
    Ready:      True  
  log-shipper:  
    State:      Running  
    Ready:      True  
Containers:
```



```
Context: kind-flap-store [RW]  
Cluster: kind-flap-store  
User: kind-flap-store  
K9s Rev: v0.51.0  
K8s Rev: v1.36.1  
CPU: n/a  
MEM: n/a  
Describe(store-dev/store-api-full)  
IP: 10.244.0.14  
IPs: 10.244.0.14  
Init Containers:  
  preparar-logs:  
    Container ID: containerd://9858c3f4a53013dffb1310c81651d0ee73b0fc6dcf330c8ccb8f4b20a63f48a  
    Image: busybox:1.36  
    Image ID: docker.io/library/busybox@sha256:73aaf090f3d85aa34ee199857f03fa3a95c8ede2ffd4cc2c2db5b94e566b11662  
    Port: <none>  
    Host Port: <none>  
    Command:  
      sh  
      -c  
      echo "$(date) [preparar-logs] arquivo de log criado" > /var/log/app/access.log  
    State:      Terminated  
    Reason:     Completed  
    Exit Code:  0  
    Started:    Fri, 18 Sep 2026 23:13:01 -0300  
    Finished:   Fri, 18 Sep 2026 23:13:01 -0300  
    Ready:      True  
    Restart Count: 0  
    Environment: <none>  
    Mounts:  
      /var/log/app from logs (rw)  
      /var/run/secrets/kubernetes.io/serviceaccount from kube-api-access-n88gk (ro)  
  log-shipper:  
    Container ID: containerd://5b2f19c12d9fc222bb8e99989d5ae082bd8cf93a86d49e3f8a4bd58d1327d9  
    Image: busybox:1.36  
    Image ID: docker.io/library/busybox@sha256:73aaf090f3d85aa34ee199857f03fa3a95c8ede2ffd4cc2c2db5b94e566b11662  
    Port: <none>
```

Figura 2 (k9s, describe): preparar-logs com State: Terminated, Reason: Completed, Exit Code: 0, o comando que cria o arquivo e o volume logs montado em /var/log/app.



```
Context: kind-flap-store [RW]
Cluster: kind-flap-store
User: kind-flap-store
K9s Rev: v0.51.0
K8s Rev: v1.36.1
CPU: n/a
MEM: n/a

Ready: True
Restart Count: 0
Environment: <none>
Mounts:
  /var/log/app from logs (rw)
  /var/run/secrets/kubernetes.io/serviceaccount from kube-api-access-n88gk (ro)
log-shipper:
  Container ID: containerd://5b2f19c12d9fc222bb8e9998d5ae082bd8cf93a86d49e3f8a4bd5c58d1327d9
  Image: busybox:1.36
  Image ID: docker.io/library/busybox@sha256:73aaf09f3d85aa34ee199857f03fa95c8ede2ffd4cc2cdb5b94e566b11662
  Port: <none>
  Host Port: <none>
  Command:
    tail
    -f
    /var/log/app/access.log
  State: Running
    Started: Fri, 18 Sep 2026 23:13:02 -0300
    Ready: True
    Restart Count: 0
    Environment: <none>
    Mounts:
      /var/log/app from logs (rw)
      /var/run/secrets/kubernetes.io/serviceaccount from kube-api-access-n88gk (ro)
Containers:
  store-api:
    Container ID: containerd://0df1b657d041e03d05b33ad88eae7e9810ea8ca75bc0b3da4f18990bca566be9
    Image: store-api:1.0
    Image ID: docker.io/library/umport-2026-09-19@sha256:d16ba0fa169507154201bc565683bbec76d43e8e6f2991ba66c39a4ec8e45851
    Port: 8000/TCP (http)
```

Figura 3 (k9s, `describe`): `log-shipper` com `State: Running`, `Ready: True`, comando `tail -F /var/log/app/access.log` e o mesmo volume `logs`.

(c) `emptyDir` compartilhado: logs do sidecar

```
$ kubectl logs store-api-full -c log-shipper -n store-dev
Sat Sep 19 02:13:01 UTC 2026 [preparar-logs] arquivo de log criado
INFO: Started server process [12]
INFO: Waiting for application startup.
INFO: Application startup complete.
INFO: Uvicorn running on http://0.0.0.0:8000 (Press CTRL+C to quit)
INFO: 10.244.0.1:47846 - "GET /ready HTTP/1.1" 200 OK
INFO: 10.244.0.1:47862 - "GET /ready HTTP/1.1" 200 OK
INFO: 10.244.0.1:34042 - "GET /ready HTTP/1.1" 200 OK
INFO: 10.244.0.1:34046 - "GET /health HTTP/1.1" 200 OK
INFO: 10.244.0.1:34050 - "GET /ready HTTP/1.1" 200 OK
```

A primeira linha foi escrita pelo `initContainer`. As demais são do `uvicorn` do container `store-api`, e quem as exibe é o `log-shipper`: os três containers enxergam o mesmo arquivo pelo volume `emptyDir`.

Figura 4 (k9s, logs): só o container `log-shipper`, exibindo as requisições atendidas pelo `store-api` (as sondas `/ready` e `/health` do kubelet).

(d) Rede compartilhada: `localhost` entre containers

```
$ kubectl exec store-api-full -c log-shipper -n store-dev -- wget -qO- http://localhost:8000/health  
{"status":"ok","host":"store-api-full"}
```

Figura 5 (k9s, shell no `log-shipper`): `wget -qO- http://localhost:8000/health` alcança o `store-api`.

Por que `localhost` funciona. Todos os containers de um Pod compartilham o mesmo network namespace: têm o mesmo IP e o mesmo espaço de portas. Logo, a porta 8000 aberta pelo `store-api` também é a porta 8000 de `localhost` para o `log-shipper`.

(e) Tráfego via Service e novas linhas no sidecar

```
$ kubectl run curl-client --image=curlimages/curl:8.10.1 -n store-dev -- sleep 3600
pod/curl-client created
$ kubectl exec curl-client -n store-dev -- sh -c 'for i in 1 2 3; do curl -s http://store-api-
full/produtos; echo; done'
[{"id":1,"nome":"Teclado","preco":199.9}]
[{"id":1,"nome":"Teclado","preco":199.9}]
[{"id":1,"nome":"Teclado","preco":199.9}]
$ kubectl logs store-api-full -c log-shipper -n store-dev --tail=5
INFO:      10.244.0.1:43564 - "GET /health HTTP/1.1" 200 OK
INFO:      10.244.0.1:43566 - "GET /ready HTTP/1.1" 200 OK
INFO:      10.244.0.15:40542 - "GET /produtos HTTP/1.1" 200 OK
INFO:      10.244.0.15:40548 - "GET /produtos HTTP/1.1" 200 OK
INFO:      10.244.0.15:40554 - "GET /produtos HTTP/1.1" 200 OK
```

As linhas `GET /produtos` vêm do IP do Pod `curl-client` (`10.244.0.15`), depois de passar pelo Service `store-api-full`.

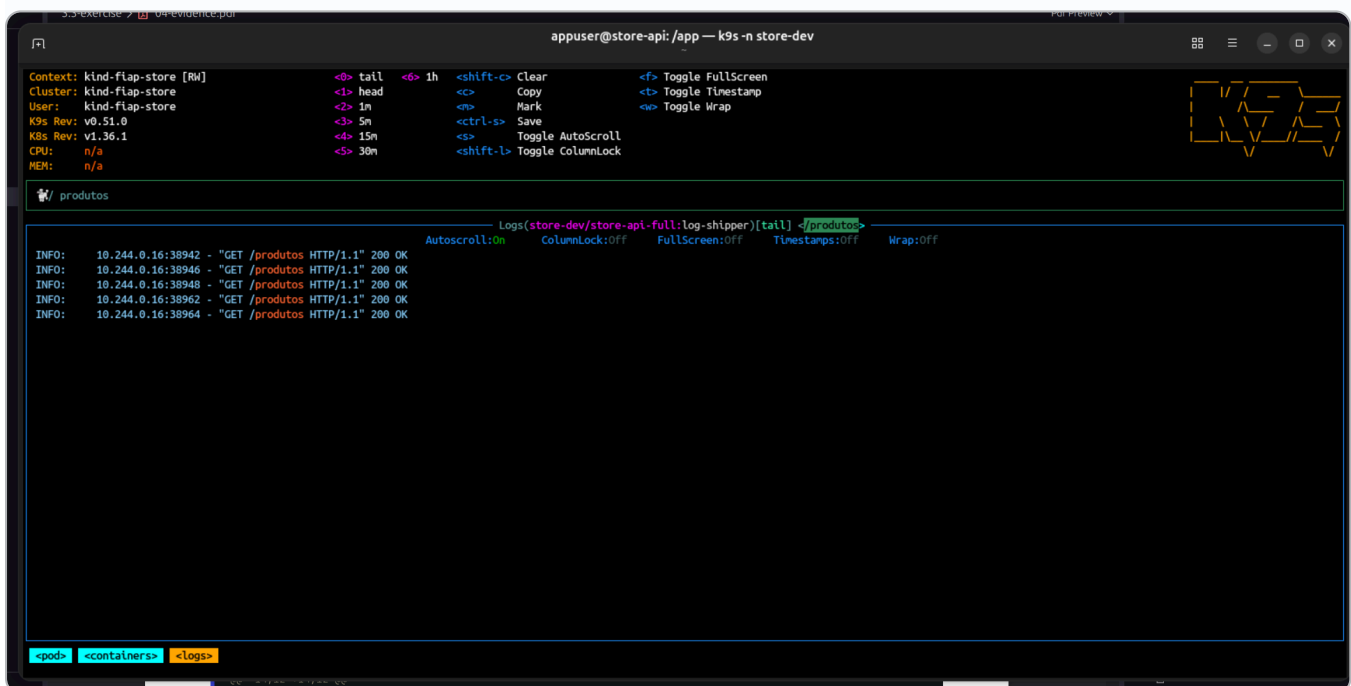


Figura 6 (k9s, logs com filtro `/produtos`): as requisições geradas via Service aparecem no `log-shipper` (IP de origem `10.244.0.16`, de uma segunda rodada de testes).

(f) Container efêmero com `kubectl debug`

O `kubectl debug` com `-it` precisa de terminal interativo. Aqui o container efêmero foi criado com um comando de espera e os comandos foram executados com `kubectl exec`. O resultado é o mesmo, e a Figura 7 repete os comandos num shell interativo do k9s.

```
$ kubectl debug store-api-full -n store-dev --image=nicolaka/netshoot --target=store-api -c debugger
-- sleep 3600
Targeting container "store-api". If you don't see processes from this container it may be because the
container runtime doesn't support this feature.
Warning: Non-root user is configured for the entire target Pod, and some capabilities granted by
debug profile may not work. Please consider using "--custom" with a custom profile that specifies
"securityContext.runAsUser: 0".
$ kubectl exec store-api-full -n store-dev -c debugger -- ss -tlnp
State Recv-Q Send-Q Local Address:Port Peer Address:PortProcess
LISTEN 0      2048          0.0.0.0:8000          0.0.0.0:*
$ kubectl exec store-api-full -n store-dev -c debugger -- curl -s localhost:8000/produtos
[{"id":1,"nome":"Teclado","preco":199.9}]
```

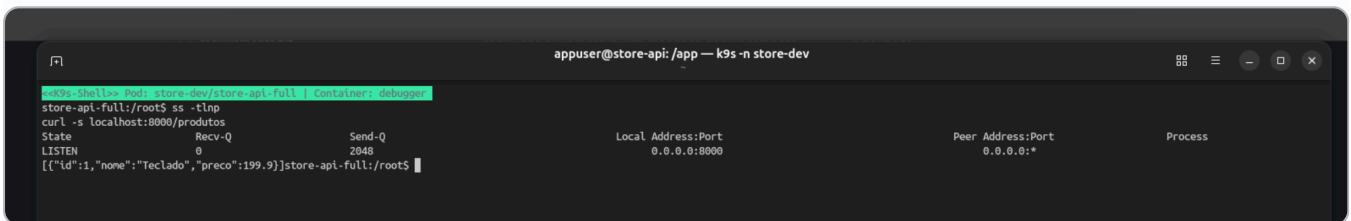


Figura 7 (k9s, shell no container efêmero `debugger`): `ss -tlnp` mostra a porta 8000 em `LISTEN` e o `curl` a `localhost:8000/produtos` responde.

O container efêmero enxerga a porta 8000 em `LISTEN` e alcança a aplicação por `localhost`, ou seja, está no mesmo network namespace do Pod, embora não faça parte do manifesto: ele foi injetado em tempo de execução no Pod em execução. A coluna `Process` do `ss` vem vazia porque o Pod inteiro roda como usuário não-root (`10001`), como o aviso do `kubectl debug` antecipa; a porta em `LISTEN` é a evidência pedida.

Pergunta de fechamento

Se este Pod virasse um `Job`, o que aconteceria com o sidecar declarado da forma antiga (em `spec.containers`)? E por que o sidecar como `initContainer` com `restartPolicy: Always` resolve?

Com o sidecar em `spec.containers`, o `tail -F` nunca termina. O Pod só chega a `Succeeded` quando **todos** os containers de `spec.containers` terminam, então o container principal concluiria a tarefa, mas o sidecar continuaria rodando e o Pod ficaria em `Running` para sempre. Como consequência, o Job nunca chegaria a `Succeeded` (ficaria ativo até estourar `activeDeadlineSeconds`, se houver). Declarado como `initContainer` com `restartPolicy: Always`, o sidecar é iniciado antes do container principal, roda durante toda a execução e, quando todos os containers regulares terminam, o kubelet o encerra automaticamente. Assim o Pod completa, o

Job chega a `Succeeded`, e o sidecar continua útil enquanto a tarefa roda (por exemplo, enviando logs até o fim).

Conclusão

O Pod `store-api-full` ficou `2/2 Running`: o `initContainer` preparou o arquivo e saiu com código 0, o sidecar `log-shipper` exibiu as linhas escritas pelo container principal (prova do `emptyDir` compartilhado), o `localhost:8000` funcionou entre containers (rede compartilhada), o tráfego via Service apareceu no sidecar e o container efêmero `nicolaka/netshoot` viu a porta 8000 em `LISTEN` no mesmo network namespace.